AquaAlarm — How It Works

This document explains the working of the AquaAlarm platform: the data model, the prediction engine, how state flows through the app, and what each dashboard does. It is written for reviewers, students, and anyone picking up the codebase.

---

1. Architecture at a glance



There is **no server and no database**. The `store.tsx` module deliberately mirrors a Convex-style backend API (reactive queries + mutation actions) so the app behaves identically to a full-stack version, but runs anywhere — a

static host such as Vercel, or a laptop with no internet. State persists across reloads via `localStorage`; if storage is unavailable (private mode) the demo keeps running in memory.

---

2. The data model (`src/lib/types.ts`)

Core entities:

Entity	Purpose
`Taluk`	One administrative taluk: current depth, critical depth, rainfall (90 d), extraction index, status, 90-day projection
`Well`	A farmer's borewell: depth, critical depth, acres, crop, phone
`User`	Demo session: role + id of the linked farmer/taluk
`Alert`	In-app early-warning message with severity and `targetRole`
`LoanRiskFlag`	Bank decision per taluk: approve / monitor / restrict + reason
`PredictionRun`	Log of each prediction-cycle run (kept to last 20)
`AppState`	The whole simulated database, in one serializable object

`src/lib/data/seed.ts` builds the initial state: 10 taluks calibrated to be credible for Tamil Nadu's hydrogeology (hard-rock western/northern taluks deeper and more stressed; delta taluks like Thanjavur shallow), three demo farmers, and default loan-risk flags per taluk.

---

3. The prediction engine (`src/lib/engine.ts`)

The engine is a simple physical projection model (simulated AI — no ML training). For each taluk:

```
net_daily_drop(ft/day) =
    extractionRateIndex × 0.0033        (extraction pressure)
  − rainfallMmLast90Days × 0.02 / 90    (monsoon recharge, spread over horizon)

depth(t)        = currentDepth + net_daily_drop × t        (t = 0…90 days)
days_to_crisis  = (criticalDepth − currentDepth) / net_daily_drop
```

Notes:

- If the net daily drop is ≤ 0 the water table is stable or rising → `daysToCrisis = null` → status **green**.

- Otherwise status is derived from `daysToCrisis`:
- **red** → crisis in under 45 days
- **yellow** → crisis in 45–90 days
- **green** → no crisis within 90 days
- The projection is sampled every 7 days for the charts.

`runPredictionCycle(state)` is the "daily cycle". It:

- Recomputes every taluk's prediction (`recalcTaluk`).
- Regenerates the per-taluk loan-risk flags.
- For every taluk now in **red**, auto-enables loan restriction and emits

three role-specific critical alerts (farmer / bank / government) — a **farmer** alert ("check your well, conserve water, switch crops"), a **bank** alert ("auto-restrict new crop loans"), and a **government** alert ("prioritise tanker allocation, borewell audit, subsidy campaign"). Duplicate alerts (same taluk + message) are not re-emitted.

- Records a `PredictionRun` summary entry.

In the UI the nav bar's **"Run prediction cycle"** button calls `runCycle()` — the manual trigger for what would be a scheduled daily job.

`resourceActionsFor(taluk)` returns the government dashboard's suggested resource actions, scaled by status (emergency tanker routes + subsidy campaign for red taluks; recharge-pond work for yellow; monitoring for green).

---

4. State management (`src/lib/store.tsx`)

- The store keeps one module-level `state: AppState` object and a set of listener callbacks.
- Components read state reactively through `useApp()`, built on `useSyncExternalStore` — so any action that replaces state re-renders every subscribed component, exactly like a reactive database query.
- Every action (`login`, `logout`, `runCycle`, `toggleAutoRestriction`, `resetDemo`) creates a **new** state object and persists it to `localStorage` under `aquaalarm-state-v1`.
- `AppProvider` hydrates persisted state on first client render. `useMounted()` distinguishes the first client render from the server render so pages never show hydration mismatches (data renders only after mount).
- `resetDemo()` wipes persisted state back to the seed data — exposed as the

footer's "Reset demo data" button.

---

5. Routing, login gate & dashboards

Route	What it does
`/`	Landing: hero, live taluk status board (counts of red taluks with crisis-day estimates), one-click entry to `/login`
`/login`	Role-based demo login: Farmer Murugan (Dharmapuri), Bank Officer, Government Official; no real auth/OTP
`/farmer`, `/bank`, `/government`	Each dashboard is wrapped in a **login gate** — see below
`/farmer`	Banner with taluk status + days to crisis; well card (current vs critical depth, days until local crisis); 12-month depth trend; recommended low-water crops ranked by % water saved vs the farmer's current crop, with subsidy match; qualifying subsidies list; 90-day taluk forecast chart; alert feed
`/bank`	Per-taluk credit-risk table (approve / monitor / restrict) with reasoning; auto-restriction toggles for red taluks; redirection of credit to water-smart loans; alert feed
`/government`	Tamil Nadu risk map (colored taluks); headline rollups; ranked high-risk taluks; per-taluk depth history + 90-day projection; resource-allocation panel (`resourceActionsFor`); alert feed

Role-scoped data is derived from the **same shared forecast** — one engine, three lenses.

Login gate (`src/components/RequireSession.tsx`)

All three dashboards are protected by a login gate. Visiting `/farmer`, `/bank`, or `/government` without an active session immediately redirects to `/login`; the dashboard content is not rendered at all until a user is signed in. Implementation details:

- `RequireSession` reads the session reactively via `useSession()` (the store hook) and calls `router.replace("/login")` in a `useEffect` when no session exists.
- It renders nothing until `useMounted()` reports the first client render, which prevents any hydration mismatch between the server render (no session) and the client (session restored from `localStorage`).
- The wrapper is used in exactly one place per dashboard — the outer JSX of each page — so the gate is trivial to extend (e.g. role checks) later.

---

6. The daily cycle, end to end

- Any user clicks **Run prediction cycle** (or the scheduled job fires).
- `runCycle()` → `runPredictionCycle()` recalculates all 10 taluks.
- Taluks whose status turned red (or is red) get auto-restriction enabled

and three new role-specific alerts.

- `PredictionRun` log entry records how many taluks were recalculated, how

many are red, and how many new alerts were emitted.

- All dashboards re-render instantly via the reactive store; the nav shows

"Last cycle: just now · N taluks · M red".

- Reloading the page restores this state from `localStorage`.

---

7. Data caveats (important)

All groundwater depths, rainfall figures, extraction indexes, and predictions are **simulated and illustrative** — calibrated to be credible for Tamil Nadu but not real CGWB/IMD readings. No real SMS, financial transactions, or loan disbursals occur. The bank view is decision-support only. This is a concept demo of the product, interface, and model.

---

8. Scripts

```
npm install        # install dependencies
npm run dev        # local dev server → http://localhost:3000
npm run build      # production build (Vercel runs this on import)
npm run lint       # eslint
npx tsx scripts/check-engine.ts   # engine sanity check (asserts statuses/geometry)
```